

Tip Text – Tip tekst

Objekat Text pruža korisne operacije za rad sa tekстом. Tip teksta ima sledeće članove:

Text.Append(text1, text2) As String

Dodaje dva tekstualna unosa. Možete koristiti operator + da biste uradili isti posao, ali ova metoda osigurava da su dva teksta uvek spojena čak i kada su dva broja, dok će operator + aritmetički sabrati ih kad god je to moguće.

Parametri:

- **text1**: Prvi deo teksta koji se dodaje. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive tipa string.

- **text2**: Drugi deo teksta koji se dodaje. Možete poslati niz da biste dodali sve njegove stavke.

Vraća: Dodati tekst koji sadrži oba navedena dela. *Primeri:*

```
X = "."
Y = "test"
TextWindow.WriteLine({
Y.Append(X),                ' test.
Text.Append(Y, X),          ' test.
"." + "1" + 4,              ' 4.1
Text.Append(".", {"1", "4"}), ' .14
X.Append({"1", "4"})        ' .14
})
```

Text.Contains(text, subText) As Boolean

Dobija da li dati tekst sadrži navedeni subText. Imajte na umu da je poređenje osetljivo na velika i mala slova.

Parametri:

- **text**: Veći tekst u kome se traži subText. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- **subText**: Podtekst koji se traži.

Vraća: Tačno ako je podtekst pronađen na bilo kojoj poziciji u datom tekstu. *Primeri:*

```
sVB = "Small Visual Basic"
TextWindow.WriteLine({
sVB.Contains("Visual"),      ' True
sVB.Contains("visual"),      ' False
Text.Contains(sVB.LowerCase, "visual"), ' True
sVB.Contains("")             ' False
})
```

Text.ConvertToLowerCase(text) As String

Konvertuje dati tekst u mala slova. Imajte na umu da je ova metoda zadržana zbog kompatibilnosti sa Small Basic-om, ali se preporučuje da se umesto nje koristi metoda ToLower, ne samo zato što je mnogo kraća, već i zato što se koristi u VB.NET-u.

Parametar:

- **text**: Tekst koji treba konvertovati u mala slova. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja (sa nazivom LowerCase) string promenljive.

Vraća: Verzija datog teksta napisana malim slovima.

Text.ConvertToUpperCase(text) As String

Konvertuje dati tekst u velika slova. Imajte na umu da je ova metoda zadržana radi kompatibilnosti sa Small Basic-om, ali se preporučuje da se umesto nje koristi metoda ToUpper, ne samo zato što je mnogo kraća, već i zato što se koristi u VB.NET-u.

Parametar:

- **text**: Tekst koji treba konvertovati u velika slova. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja (sa imenom UpperCase) string promenljive.

Vraća: Verzija datog teksta napisana velikim slovima.

Text.EndsWith(text, subText) As Boolean

Dobija da li se dati tekst završava navedenim subText-om. Poređenje je osetljivo na velika i mala slova.

Parametri:

- **text**: Veći tekst u kome se traži subText. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metod proširenja string promenljive.

- **subText**: Podtekst koji treba tražiti.

Vraća: True ako je podtekst pronađen na kraju datog teksta. *Primeri:*

```
sVB = "Small Visual Basic"
TextWindow.WriteLine({
    sVB.Contains("Basic"),           ' True
    sVB.Contains("basic"),           ' False
    Text.Contains(sVB.LowerCase, "basic"), ' True
    sVB.Contains("")                 ' False
})
```

Text.Format(text, values) As String

Formatira string zamenom rezervisanih mesta [1], [2], ... [n] odgovarajućim stavkama iz datog niza vrednosti.

Parametri:

- **text**: String za formatiranje. Koristite rezervisana mesta [1], [2],... [n] u stringu, da biste referencirali vrednosti[1], vrednosti[2], ... vrednosti[n]. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive tipa string.

- **values**: Niz čiji će elementi biti korišćeni za zamenu rezervisanih mesta [1], [2],... [n] ako se nalaze u tekstu.

Vraća: Formatirani string nakon zamene rezervisanih mesta [1], [2],... [n] elementima iz niza vrednosti. *Primer:*

```
Student = "[1]. Ime: [2], Godine: [3]."
```

```
TextWindow.WriteLine({
    Text.Format(Student, {1, "Aca", 15}),
    Student.Format({2, "Nikola", 10}),
    Student.Format({3, "Jasmina", 12})
})
```

Text.GetCharacter(characterCode) As String

Dobija odgovarajući znak za dati Unicode kôd znaka, koji se zatim može koristiti sa običnim tekstom.

Parametar:

- *characterCode*: Kod znaka (baziran na Unicode-u) za traženi znak.

Vraća: Unikod znak koji odgovara navedenom kodu.

Primeri: Dodajte tekstualno polje i dugme na obrazac, dvaput kliknite na dugme i napišite ovaj kod u funkciji za obradu događaja Button1_OnClick:

```
For code = 32 To 1000  
c = Text.GetCharacter(code)  
TextBox1.Append(code)  
TextBox1.Append(" = ")  
TextBox1.AppendLine(c)  
Next
```

Pokrenite program i kliknite na dugme. Ovo će ispisati 1000 Unikod znakova u tekstualnom polju, osim prvih 31 znaka, jer je većina njih kontrolni znak koji se ne može ispisati. Kod 32 predstavlja znak razmaka, i možete videti šta sledeći kodovi predstavljaju!

Možete odštampati još mnogo karaktera, ali vodite računa da će dodavanje hiljada redova u polje za tekst biti sporo i beskorisno jer ne možete pročitati hiljade redova. Zato vam preporučujem da pregledate opsege karaktera, gde svaki opseg sadrži samo 100 karaktera. Možete lako dodati polje za tekst da biste uneli početak opsega i izmeniti gornji kôd tako da petlja for počinje od početka opsega i završava se na početku opsega + 99.

Text.GetCharacterAt(text, poz) Kao string

Dobija karakter koji postoji na datoj poziciji u tekstu. Imajte na umu da možete direktno koristiti indeks `text[poz]` da biste dobili karakter.

Parametri:

- *text*: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- *poz*: Pozicija karaktera.

Vraća: Karakter je postojao na datoj poziciji ili prazan string "" ako je pozicija nevažeća.

Primeri: U sledećem kodu, pozicije -1, 0 i 5 su nevažeće, pa će metoda `GetCharacterAt` vratiti prazan string "" (i ništa neće biti napisano u tekstualnom prozoru u odgovarajućim redovima), dok su pozicije 1 i 4 validne, pa će metoda `GetCharacterAt` vratiti T i t. Takođe smo koristili indeks da bismo dobili e i s na pozicijama 2 i 3:

```
X = "Test"  
TextWindow.WriteLine({  
Text.GetCharacterAt(X, -1),  
X.GetCharacterAt(0),  
X.GetCharacterAt(1),    ' T  
X[2],                  ' e  
X[3],                  ' s  
X.GetCharacterAt(4),    ' t  
X.GetCharacterAt(5)  
})
```

Text.GetCharacterCode(character) As Double

Dobija odgovarajući kôd znaka za dati Unicode znak.

Parametar:

• *character*: Znak čiji se kôd traži.

Vraća: Kôd zasnovan na Unicode-u koji odgovara navedenom znaku.

Primer: Unicode 100000 je predstavljen parom surogata.

Ne moramo biti svesni toga kada koristimo metode `GetCharacter` i `GetCharacterCode` da bismo izvršili konverzije. Sledeći primeri pokazuju da će konvertovanje Unicode 100000 u znak vratiti string dužine 2, a ovaj string možete poslati nazad metodi `GetCharacterCode` da biste ponovo dobili Unicode 100000!

```
x = Text.GetCharacter(100000)
TextWindow.WriteLine(x.Length) '2
c = Text.GetCharacterCode(x)
TextWindow.WriteLine(c)        ' 100000
```

Text.IndexOf(text, subText, start, isBackward) As Double

Pronalazi poziciju gde se podtekst pojavljuje u navedenom tekstu. Pretraga je osetljiva na velika i mala slova.

Parametri:

- *text*: Tekst u kojem se traži subText. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.
- *subText*: Podtekst koji treba pretraživati.
- *start*: Pozicija teksta od koje treba početi pretragu.
- *isBackward*: Koristite True ako želite da pretražujete od početka do prve pozicije u tekstu (= 1), ili False ako želite da idete napred do kraja teksta.

Vraća: Pozicija na kojoj se podtekst pojavljuje u navedenom tekstu, ili 0 ako tekst ne postoji.

Primer:

```
SVB = "Small Visual Basic"
TextWindow.WriteLine({
SVB.IndexOf("Visual", 1, False),    ' 7
SVB.IndexOf("visual", 1, False),    ' 0
Text.IndexOf(
SVB.LowerCase, "visual", 1, False), ' 7
SVB.IndexOf("s", 2, False),         ' 9
SVB.IndexOf("s", SVB.Length, True) ' 16
})
```

Text.GetLength(text) As Double

Dobija broj znakova u datom tekstu.

Parametar:

- *text*: Tekst čija je dužina potrebna. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja (sa imenom `Length`) string promenljive.

Vraća: Dužina datog teksta. *Primeri*:

```
X = "Test"
TextWindow.WriteLine({
Text.GetLength(""),    ' 0
Text.GetLength(X),     ' 4
X.Length               ' 4
})
' Ispisuje sve tekstualne znakove
For I = 1 To X.Length
TextWindow.WriteLine(X[I])
Next
```

Ili jednostavno koristite `ForEach`:

```
ForEach C In X
TextWindow.WriteLine(C)
Next
```

Text.GetSubText(text, start, length) As String

Dobija podtekst iz datog teksta.

Parametri:

- *text*: Tekst iz kojeg se izvodi podtekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja (sa imenom SubText) string promenljive.
- *start*: Početna pozicija podteksta. Ako je početak manji od 1, podtekst će početi od prvog karaktera teksta. Ako je početak veći od dužine teksta, ova metoda će vratiti prazan string "".
- *length*: Određuje dužinu podteksta. Ako je dužina manja od 1, ova metoda će vratiti prazan string "". Ako dužina podteksta prelazi dužinu teksta, dodatna dužina će biti ignorisana i podtekst će se zaustaviti na kraju teksta.

Vraća: Traženi podtekst. *Primeri*:

```
SVB = "Small Visual Basic"
TextWindow.WriteLine({
Text.GetSubText(SVB, 1, 5), ' Small
SVB.SubText(7, 6), ' Visual
SVB.SubText(14, 5), ' Basic
SVB.SubText(14, 100), ' Basic
SVB.SubText(14, 0),
SVB.SubText(-1, SVB.Length),
SVB.SubText(100, 5) ' Small Visual Basic
})
```

Text.GetSubTextToEnd(text, start) As String

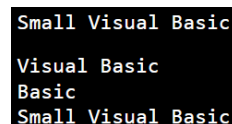
Dobija podtekst iz datog teksta od navedene pozicije do kraja.

Parametri:

- *text*: Tekst iz kojeg treba izvesti podtekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja (sa imenom SubTextToEnd) string promenljive.
- *start*: Početna pozicija podteksta. Ako je početak manji od 1, podtekst će početi od prvog karaktera teksta. Ako je početak veći od dužine teksta, ova metoda će vratiti prazan string "".

Vraća: Traženi podtekst. *Primeri*:

```
SVB = "Small Visual Basic"
TextWindow.WriteLine({
Text.GetSubTextToEnd(SVB, 1),
SVB.SubTextToEnd(100),
SVB.SubTextToEnd(7),
SVB.SubTextToEnd(14),
SVB.SubTextToEnd(-1)})
```



```
Small Visual Basic
Visual Basic
Basic
Small Visual Basic
```

Text.IsEmpty(text) As Boolean

Vraća True ako je dati tekst prazan, ili False u suprotnom. Imajte na umu da možete direktno koristiti operator = da biste uporedili tekst sa praznim stringom "".

Parametar:

- *text*: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja string promenljive.

Vraća: True ako je tekst prazan, ili False u suprotnom.

Text.IsNumeric(text) As Boolean

Proverava da li string sadrži numeričku vrednost.

Parametar:

- *text*: String za proveru njegove vrednosti. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja string promenljive.

Vraća: True ako je tekst broj, ili False u suprotnom. *Primeri*:

```
X = "1.4"
Y = "100$"
TextWindow.WriteLine({
X.IsNumeric,           ' True
Y.IsNumeric,           ' False
Text.IsNumeric(""),    ' False
Text.IsNumeric(-0.6),  ' True
Text.IsNumeric("Jedan") ' False
})
```

Text.IsSubText(text, subText) As Bulean

Dobija vrednost da li je dati podtekst podskup većeg teksta. Slično je metodi Text.Contains.

Parametri:

- *text*: Veći tekst u kome će se pretraživati podtekst.

- *subText*: Podtekst koji treba pretraživati.

Vraća: True ako je podtekst pronađen u datom tekstu, ili False u suprotnom. *Primeri*:

```
SVB = "Small Visual Basic"
TextWindow.WriteLine({
Text.IsSubText(SVB, "Visual"),      ' True
Text.IsSubText(SVB, "visual"),      ' False
Text.IsSubText(SVB.LowerCase, "visual"), ' True
Text.IsSubText(SVB, "")             ' False
})
```

Text.LowerCase As String

Ovo je svojstvo proširenja string promenljivih. Vraća novi tekst od trenutnog teksta čiji su svi znakovi ispisani malim slovima. Takođe možete koristiti metod Text.ToLower da biste obavili isti posao.

Text.Length As String

Ovo je svojstvo proširenja string promenljivih. Vraća dužinu trenutnog teksta. Takođe možete koristiti metod Text.GetLength da biste obavili isti posao.

Text.NewLine As String

Vraća znakove novog reda (Chars.Cr + Chars.Lf) tako da ih možete dodati tekstu da biste nastavili pisanje u novom redu. Ovo svojstvo je slično svojstvu Chars.CrLf.

Text.PadLeft(text, totalWidth) As String

Dodaje razmake (ako je potrebno) levo od datog teksta, tako da njegova dužina bude jednaka datoj ukupnoj širini. Ovo je korisno kada želite da poravnate tekstove desno.

Imajte na umu da ako je dužina ulaznog teksta veća ili jednaka datoj dužini, biće prikazan kakav jeste i neće biti skraćen.

Parametri:

- **text**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- **totalWidth**: Željena ukupna dužina teksta.

Vraća: Novi tekst koji ima datu ukupnu dužinu (barem). *Primeri:*

```
X = "abcde"
TW.WriteLine({
  "|" & Text.PadLeft("a", 4) & "|",
  "|" & Text.PadLeft("ab", 4) & "|",
  "|" & Text.PadLeft("abc", 4) & "|",
  "|" & Text.PadLeft("abcd", 4) & "|",
  "|" & X.PadLeft(4) & "|"
})
```

```
| a |
| ab |
| abc |
| abcd |
| abcde |
```

Text.PadRight(text, totalWidth) As String

Dodaje razmake (ako je potrebno) desno od datog teksta, tako da njegova dužina bude jednaka datoj ukupnoj širini. Ovo je korisno kada želite da poravnate tekstove levo.

Imajte na umu da ako je dužina ulaznog teksta veća ili jednaka datoj dužini, biće prikazan kakav jeste i neće biti skraćen.

Parametri:

- **text**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive tipa string.

- **totalWidth**: Željena ukupna dužina teksta.

Vraća: novi tekst koji ima datu ukupnu dužinu (barem). *Primeri:*

```
X = "abcde"
TW.WriteLine({
  "|" & Text.PadRight("a", 4) & "|",
  "|" & Text.PadRight("ab", 4) & "|",
  "|" & Text.PadRight("abc", 4) & "|",
  "|" & Text.PadRight("abcd", 4) & "|",
  "|" & X.PadRight(4) & "|"
})
```

```
| a |
| ab |
| abc |
| abcd |
| abcde |
```

Text.Repeat(text, count) As String

Ponavlja uneti tekst zadati broj puta.

Parametri:

- **text**: Uneti tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja promenljive tipa string.

- **count**: Broj puta ponavljanja teksta.

Vraća: Ponovljeni tekst. *Primeri:*

```
X = "abc"
TW.WriteLine({
  Text.Repeat("a", 4),
  Text.Repeat("ab", 4),
  Text.Repeat(5, 5),
  X.Repeat(3)
})
```

```
aaaa
abababab
55555
abcabcabc
```

Text.SetCharacterAt(text, pos, newText) As String

Menja karakter koji postoji na datoj poziciji u dati novi tekst. Imajte na umu da možete direktno koristiti indekse da biste postavili novi tekst:


```
text[pos] = newText
```

ali u ovom slučaju, ulazni tekst će biti direktno pogođen promenom. U stvari, gornja izjava je ekvivalentna:

```
text = Text.SetCharacterAt(text, pos, newText)
```

Parametri:

- **text**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- **pos**: Pozicija karaktera: Koristite 0 da biste umetnuli novi tekst pre prvog karaktera. Koristite negativan broj da biste umetnuli novi tekst pre prvog karaktera sa više razmaka dodatih između njih. Koristite broj veći od dužine teksta da biste dodali novi tekst posle poslednjeg karaktera, sa dodatnim razmacima između njih da biste dostigli datu poziciju.

- **newText**: Novi tekst koji treba postaviti na datu poziciju. Pošaljite prazan string "" da biste uklonili trenutni karakter iz teksta ili pošaljite jedan ili više karaktera da biste ga zamenili.

Vraća: Novi tekst sa karakterom promenjenim na datu vrednost. Uneti tekst se neće promeniti.

Primeri:

```
X = "test"
TextWindow.WriteLine({
Text.SetCharacterAt(X, -1, "a"), 'test
X.SetCharacterAt(0, "a"),       'atest
Text.SetCharacterAt(X, 1, "a"), 'aest
X.SetCharacterAt(2, "..e"),     't..est
X.SetCharacterAt(5, "s"),       'tests
X.SetCharacterAt(6, "#1")       'test #1
})
```

```
a test
atest
aest
t..est
tests
test #1
```

Ili koristite indekser da uradite isto:

```
X = "test"
X[-1] = "a"
TextWindow.WriteLine(X) 'test
X = "test"
X[0] = "a"
TextWindow.WriteLine(X) 'atest
X = "test"
X[1] = "a"
TextWindow.WriteLine(X) 'aest
X = "test"
X[2] = "..e"
TextWindow.WriteLine(X) 't..est
X = "test"
X[5] = "s"
TextWindow.WriteLine(X) 'tests
X = "test"
X[6] = "#1"
TextWindow.WriteLine(X) 'test #1
```

```
a test
atest
aest
t..est
tests
test #1
```

Text.Split(text, separator, trim, removeEmpty) As Array

Deli dati tekst na datom separatoru.

Parametri:

- **text**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- **separator**: Jedan ili više znakova na kojima treba podeliti tekst. Separator se neće pojaviti u rezultatu. Takođe možete poslati niz da biste koristili njegove stavke kao separatore.

- **trim**: Koristite True da biste uklonili razmake (razmake, tabulatore i znakove reda) sa početka

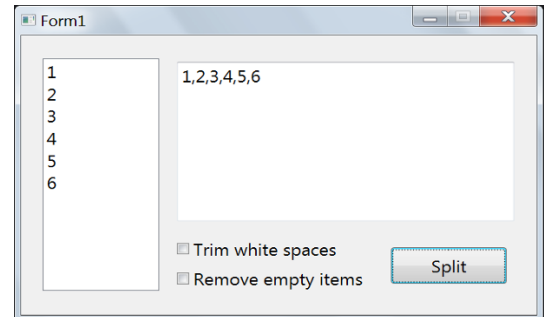
i kraja razdvojenih nizova.

- **removeEmpty**: Koristite True da biste uklonili prazne stavke iz rezultujućeg niza. Prazne stavke se mogu pojaviti ako u tekstu postoje dva uzastopna separatora bez ičega između, ili zato što stavka sadrži razmake i vi izaberete da ih skratite.

Vraća: Niz koji sadrži rezultujuće stavke.

Primer: Možete videti ovu metodu u akciji u projektu „Split Text“ – *Podeli tekst* u folderu samples. Forma omogućava korisniku da upiše stavke u tekstualno polje odvojene zareзима ili novim redovima, a zatim klikne na dugme za podelu da bi podelio stavke i dodao ih u okvir sa listom.

Forma takođe prikazuje dva polja za potvrdu koja omogućavaju korisniku da izabere da li će skratiti razmake i ukloniti prazne stavke. Sledeća slika prikazuje ovaj grafički korisnički interfejs:



Rukovalac klikova na dugme za razdvajanje jednostavno poziva metodu Text.Split da bi dobio niz rezultata i dodaje ga na listu koristeći metodu AddItem:

```
Sub Button1_OnClick()  
  ListBox1.RemoveAllItems()  
  ListBox1.AddItem(  
    Text.Split(  
      TextBox1.Text,  
      {",", Chars.CrLf},  
      ChkTrim.Checked,  
      ChkNoEmpty.Checked  
    )  
  )  
EndSub
```

Text.StartsWith(text, subText) As Boolean

Dobija vrednost da li dati tekst počinje navedenim subText-om. Poređenje je osetljivo na velika i mala slova.

Parametri:

- **text**: Veći tekst u kome se traži. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

- **subText**: Podtekst koji se traži.

Vraća: True ako je podtekst pronađen na početku datog teksta. *Primeri*:

```
sVB = "Small Visual Basic"  
TextWindow.WriteLine({  
  sVB.StartsWith("Small"), ' True  
  sVB.StartsWith("small"), ' False  
  Text.StartsWith(sVB.LowerCase, "small"), ' Tačno  
  sVB.StartsWith("") ' False  
})
```

True
False
True
False

Text.ToArray(text) As Array

Možda nikada nećete morati da koristite ovu metodu, jer sVB može automatski da rekonstruiše nizove iz njihove string reprezentacije.

Jedini izuzetak može da se desi kada koristite metode Array.Append i Array.SetItemAt, koje obrađuju reference nizova velikih nizova u for petljama kako bi bile veoma brze u poređenju sa

korišćenjem `inxeder-a`, što je retka situacija i ne očekuje se u obrazovnim primerima.

U takvim slučajevima, ako imate string reprezentaciju niza, ne možete je direktno proslediti tim metodama i morate prvo koristiti metod `Text.ToArray` da biste konvertovali text u niz.

Text.ToLower(text) As String

Konvertuje sve znakove ulaznog teksta u mala slova.

Parametar:

- **text**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja (sa imenom `LowerCase`) string promenljive.

Vraća: Tekst napisan malim slovima. *Primeri*:

```
X = "aBC"
TextWindow.WriteLine({
X.LowerCase,          ' abc
Text.ToLower(X)       ' abc
})
```

Text.ToNumber(text) As Double

Konvertuje ulazni tekst u broj. Kompilator koristi ovu metodu da bi osigurao da su promenljive `For` petlje (brojač, početak, kraj i korak) uvek validni brojevi. Ne morate sami da koristite ovu metodu, jer ona konvertuje pojedinačne znakove u njihov Unicode kôd. Umesto toga, trebalo bi da koristite metod `Text.IsNumeric` da biste proverili da li promenljiva sadrži važeći broj pre nego što je upotrebite.

Parametar:

- **tekst**: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

Vraća: Ako je tekst broj, vraća svoju numeričku vrednost. Ako tekst sadrži samo jedan nenumerički znak, vraća njegov ASCII kod. U suprotnom, vraća 0.

Primeri: Sledeća `for` petlja počinje od "a" do "e". sVB kompajler će pozvati `Text.ToNumber("a")` i `Text.ToNumber("e")` da bi konvertovao ova dva niza u brojeve, što će dobiti Unicode dva slova a i e. Dakle, možemo da ispišemo znakove od a do e samo vraćanjem znaka koji predstavlja brojač I:

```
For I = "a" To "e"
TextWindow.WriteLine(Text.GetCharacter(I))
Next
```

a
b
c
d
e

Text.ToString(value) As String

Konvertuje datu vrednost ili niz u string.

Parametar:

- **value**: Ulazna vrednost. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

Vraća: String reprezentacija ulazne vrednosti. Na primer, niz stringova može biti oblika {1, 2, 3}. *Primeri*:

```
X = {"a", "b", "c"}
TextWindow.WriteLine({
X,          ' 1=a;2=b;3=c;
Text.ToString(X) ' {a, b, c}
})
```

1=a;2=b;3=c;
{a, b, c}

Text.ToUpper(text) As String

Konvertuje sve znakove ulaznog teksta u velika slova.

Parametar:

- *text*: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao svojstvo proširenja (sa nazivom UpperCase) string promenljive.

Vraća: Tekst napisan velikim slovima. *Primeri*:

```
X = "aBc"
TextWindow.WriteLine({
X.UpperCase,          ' ABC
Text.ToUpper(X)      ' ABC
})
```

Text.Trim(text) As String

Uklanja sve vodeće i završne znakove razmaka iz datog teksta. Znakovi razmaka uključuju razmake, tabulatore i simbole linije.

Parametar:

- *text*: Ulazni tekst. Ovaj argument se može izostaviti ako pozovete ovu metodu kao metodu proširenja string promenljive.

Vraća: Skraćeni string. *Primeri*:

```
X = " A "
TextWindow.WriteLine({
|" " + X + " |",      ' | A |
|" " + X.Trim() + " |",  ' |A|
|" " + Text.Trim(X) + " |"  ' |A|
})
```



A visual representation of the string "A" with leading and trailing spaces. It shows three rows: the first row is " | A |", the second row is " |A|", and the third row is " |A|". This illustrates how the Trim method removes the leading and trailing spaces from the string.

Text.UpperCase As String

Ovo je svojstvo proširenja string promenljivih. Vraća novi tekst čiji su svi znakovi velika slova trenutnog teksta. Takođe možete koristiti metod Text.ToUpper da biste uradili isti posao.